

Wegen NFT Trait Store — Full Rebuild Blueprint

This document is a complete specification of the "Wegen NFT Trait Store" application, written so that another AI coding tool (with zero access to this project) can rebuild it from scratch on a different platform/stack. It documents what exists in the current codebase as of this writing — not aspirational features.

1. Plain-English Summary

Wegen NFT Trait Store is a Web3 marketplace where owners of "Wegen" (and "Wegenette") NFT collections buy cosmetic **traits** (background, body, clothes, eyes, headgear, mouth) and apply them to their NFTs, similar to skins/cosmetics in a game.

Who it's for: - **NFT holders / collectors** who want to customize the look of their NFT by buying traits, trying them on in a sandbox, trading with other holders, or completing bounties/quests to earn points and redeem exclusive rewards. - **The project team / admins** who need to create and price traits, manage supply, monitor sales, and configure store-wide settings and appearance — all without needing engineering support for routine changes.

Problem it solves: NFT collections often want to let holders customize/evolve their NFT's appearance and add ongoing utility/engagement (trait drops, swap markets, point economies, quests) without building bespoke tooling for every drop.

Core loop: connect wallet → sign a message (proves wallet ownership, no gas) → browse/buy traits with a simulated ETH transaction → equip traits onto an owned NFT → optionally "Save On Chain" to persist the current look as authoritative metadata → trade extras via swap/market → earn points via purchases/bounties → redeem points for exclusive rewards.

2. Complete Feature List

Storefront / buyer-facing

- Browse traits by category and theme, with rarity badges (common/uncommon/rare/legendary)
- Live NFT preview banner: pick one of your owned NFTs and see how a hovered/selected trait would look before buying
- Shopping cart (add/remove trait, quantity), checkout flow with a simulated blockchain confirmation dialog
- "Legends" store mode: browse and buy variant "skins" for special 1-of-1 Legend items
- Store stats (□ traits, categories, purchases, holders) shown on the storefront
- Store maintenance mode + wallet allowlist gate (config served from `/api/store/config`)
- Two collections supported side-by-side: "Wegens" and "Wegenettes", switchable via a collection selector, each with its own color theme, trait catalog, and NFT set

Trait Locker ("Stash")

- View all purchased traits owned by the connected wallet
- Filter/sort by layer(category), rarity, purchase date, name
- Drag-and-drop (or button) to equip a trait onto a selected owned NFT
- Un-equip a trait back into the locker
- "Save On Chain" (SOC): commits the current equipped loadout as a metadata update with a simulated tx hash

My Wegens / NFTs page

- Gallery of the wallet's owned NFTs showing currently equipped traits per category
- Dialog to pick a locker trait to equip into a given category slot
- Remove an equipped trait (returns it to locker)
- Re-open the SOC dialog to (re)commit metadata on-chain, paying an optional ETH "on-chain update fee"

Trait Swap & Market

- **Swap:** post an open listing offering one or more locker traits, describing what you want in return ("looking for" free text); other users propose a trade by offering their own locker traits; poster/acceptor items get swapped atomically
- **Market:** list a locker trait for sale at a fixed ETH price; another wallet can buy it outright; ownership transfers in `locker_items`
- Cancel your own swap/market listings

Sandbox ("fitting room")

- Try on any trait (owned or not) purely visually, randomize a full look, clear selection
- Switch between "original" and alternate variant packs/skins per trait
- Save current composition as a downloaded PNG (canvas composite)
- Daily bounty: match a specific target look to trigger a celebration animation and earn bonus points (rate-limited per day)

My Legends

- View owned "Legend" (1-of-1) NFTs
- Toggle between enabled cosmetic variants per legend
- Link out to OpenSea for the NFT

Bounties / Points economy

- Global leaderboard (top wallets by points)
- Personal stats: total points, rank, daily bounty completions, point transaction history
- Claim pending/airdropped points
- Redeem points for exclusive "bounty traits" (optionally linked to a real vault trait, delivering an actual locker item instead of a cosmetic-only record)

Bundles & Points purchases

- Buy "Store Points" in packs (USD-priced, ETH-charged) as a secondary currency

- Buy "Trait Bundles" — a curated multi-trait pack sold as one purchase, unpacked into individual locker items
- Search/filter bundles

Admin panel (/admin)

- **Traits:** create/edit/delete, set name/category/theme/description/image/media type (image/gif/video/audio)/price (USD, auto-converts to ETH from live rate)/supply/rarity/active flag/payout splits (revenue-share wallets summing to 100%)
- **Bulk variant upload:** drag-and-drop many files at once, auto-matched to existing traits by filename, grouped into a named "variant pack"
- **Variant packs manager:** enable/disable whole packs or individual variants per collection
- **Legends:** create/edit/delete legend NFTs and their variants
- **NFTs:** list/delete NFT records (admin view of `wegen_nfts`)
- **Fees:** buying/selling fee %, marketplace listing fee %, on-chain update ETH fee + payout wallets
- **Store settings:** name, tagline, open/closed toggle, announcement banner, max traits per order, contract address, collection wallet, network name, social links, maintenance mode + wallet whitelist, ineligible NFT list
- **Appearance:** logo/background/banner image uploads, custom color tokens (primary/secondary/text/etc.)
- **Layer order:** drag to reorder which trait category renders on top in composites
- **Stats dashboard:** total traits/sales/revenue/active/out-of-stock, top-selling traits, recent activity feed
- **Transactions log:** paginated, filterable by type (buy/sell/trade)
- **Points & Bounties:** manage point packs, manage bounty reward traits, airdrop points to one/many/all wallets, view leaderboard and point transaction log
- **Bundles:** create/edit/delete trait bundles and their contained traits

Wallet / auth

- Multi-wallet detection: MetaMask, Phantom, Backpack, Coinbase Wallet, OKX, Trust Wallet, Rabby, Rainbow, Brave, generic injected — EVM-only end to end (no separate Solana flow)

- Sign-In With Ethereum (SIWE)-style challenge/response, session cookie on success
 - Auto-restores session on page load; listens for account/chain changes and disconnects on account switch
 - Retries once on address-mismatch signing errors (handles wallets that report stale accounts)
-

3. Page-by-Page Breakdown

/ — Store

- **Purpose:** main marketplace for individual traits and Legend variant packs.
- **Data shown:** trait grid (image, name, category, rarity badge, USD/ETH price, remaining supply), category/theme filter tabs, store stats, NFT preview banner (if wallet connected), cart drawer.
- **Actions:** hover/click a trait to preview on selected NFT; add to cart; remove from cart; clear cart; switch store mode (Traits vs Legends); switch collection (Wegens/Wegenettes); connect wallet; checkout (confirms each cart item via a simulated tx dialog).
- **After checkout:** `POST /api/locker/:walletAddress/purchase` per item → decrements trait supply, inserts a `locker_items` row, logs a `transactions` row, awards 25 points per unit; UI invalidates store-stats and locker caches, shows success toast, empties cart.

/locker — Trait Locker

- **Purpose:** manage owned traits and equip them to an NFT.
- **Data shown:** owned locker items with equip status, owned NFTs to select as the equip target, current variant overrides.
- **Actions:** select NFT; filter/sort locker items; drag or click "Equip" on an item; click "X"/"Remove" on an equipped trait; open "Save On Chain" confirmation.
- **After actions:** Equip → `POST /api/nfts/:tokenId/apply-trait` (auto-unequips any existing trait in the same category first). Remove → `POST /api/nfts/:tokenId/remove-trait`. SOC → `POST /api/nfts/:tokenId/confirm-traits` (writes `metadataTxHash` / `metadataUpdatedAt` , optional `variantPack` , awards 25 points).

`/nfts` — My Wegens

- **Purpose:** NFT-centric view of equipped traits per owned token, with re-confirm/save-on-chain flow.
- **Data shown:** each owned NFT card with its equipped traits by category, on-chain update ETH fee (from `/api/admin/fees`), variant collections available.
- **Actions:** open "equip" dialog (tabs per category, pick from locker); remove an equipped trait; open SOC dialog and confirm (may show an ETH fee before saving).
- **After actions:** same endpoints as Locker (`apply-trait` , `remove-trait` , `confirm-traits`); success toast shows the returned tx hash.

`/swap` — Trait Swap & Market

- **Purpose:** peer-to-peer trading (barter) and fixed-price resale of locker traits.
- **Data shown:** open swap listings (with offered items and "looking for" text), active market listings (with ETH price), mini-locker of the connected wallet for building offers.
- **Actions:** "Post a Swap" (pick items + looking-for text); "Propose Trade" on someone else's swap (pick your own items to offer); "Buy" a market listing; "Cancel" your own listing; list a locker item on the market.
- **After actions:** Swap accept → `POST /api/swap/listings/:id/accept` (atomically reassigns `walletAddress` on both sides' `locker_items` , marks listing `accepted` , logs `trade` transactions). Market buy → `POST /api/market/listings/:id/buy` (reassigns item ownership, marks listing `sold` , logs `sale` transactions for both parties).

`/sandbox` — Sandbox / Fitting Room

- **Purpose:** try any trait visually (no ownership required) and attempt the daily bounty look.
- **Data shown:** full trait catalog by category, active daily bounty target (if enabled), variant pack tabs.
- **Actions:** pick/clear a trait per category; randomize; save composite as PNG download; attempt to match bounty look.

- **After actions:** On bounty match → `POST /api/bounties/sandbox-complete` (rate-limited per day via `daily_bounty_completions`), awards points, triggers celebration UI.

`/my-legends` — My Legends

- **Purpose:** view and re-skin owned 1-of-1 Legend NFTs.
- **Data shown:** owned legends (`GET /legends/mine`), enabled variants per legend.
- **Actions:** pick a variant to preview/display; refresh list; open OpenSea link.
- **After actions:** purely client-side display state change (no persistence call for variant selection).

`/bounties` — Bounties & Rewards

- **Purpose:** points economy hub.
- **Data shown:** global leaderboard, personal rank/points/history/pending, reward traits redeemable with points.
- **Actions:** "Claim Pending" points; "Redeem" a reward trait.
- **After actions:** Claim → `POST /api/bounties/claim-points` . Redeem → `POST /api/bounties/traits/:id/redeem` (deducts points; if the reward trait is vault-linked, also creates a real `locker_items` row — otherwise a cosmetic-only reward record).

`/bundles-points` — Packs & Points

- **Purpose:** buy Store Points currency and multi-trait Bundles directly.
- **Data shown:** point packs (USD value → points granted), trait bundles (contained traits, price), current point balance.
- **Actions:** search/filter bundles; "Buy Points" (opens confirm modal); "Buy Bundle" (opens confirm modal).
- **After actions:** Points → `POST /api/points/purchase` (adds to `store_points` , logs `store_point_purchases`). Bundle → `POST /api/bundles/:bundleId/purchase` (decrements bundle supply, inserts one `locker_items` row per contained trait, logs `bundle_purchases`).

`/admin` — Admin Panel

- **Purpose:** back-office management for traits, legends, NFTs, fees, store settings, appearance, points/bounties, bundles, and analytics.

- **Data shown:** tabbed dashboard — Traits table, Legends table, NFTs table, Stats cards + charts, Transactions log, Fees form, Store Settings form, Appearance form, Layer order list, Point Packs table, Bounty Traits table, Bundles table, Leaderboard, Point log.
 - **Actions:** create/edit/delete for every entity above; bulk variant upload; toggle variant packs; drag-reorder layers; airdrop points; view paginated logs.
 - **After actions:** standard CRUD calls to the `/admin/*` endpoints below; most mutations invalidate the relevant React Query cache keys and show a toast.
 - ⚠ **No authentication is enforced on this route or its API endpoints today** (see Section 9).
-

4. User-Flow Breakdown by Role

New visitor (no wallet connected)

1. Lands on `/` and can browse traits, view store stats, switch collections — fully read-only.
2. Clicking any wallet-gated action (buy, equip, locker, NFTs, bounties) prompts a "Connect Wallet" call.
3. Connect flow: detect installed wallets → user picks/only one is available → `eth_requestAccounts` → fetch nonce (`GET /api/auth/nonce`) → sign SIWE message → `POST /api/auth/verify` → session cookie set → `isConnected` becomes true app-wide.

Wallet-connected holder

1. After connecting, `/locker` and `/nfts` unlock, showing their owned traits/NFTs.
2. Buys traits on `/` → items land in `/locker`.
3. Equips traits from `/locker` or `/nfts` onto a chosen NFT; can re-equip/un-equip freely until "Saved On Chain".
4. Optionally trades extras via `/swap` (barter) or lists on the market (fixed ETH price).
5. Visits `/sandbox` to preview looks risk-free and chase the daily bounty for bonus points.

6. Checks `/bounties` for rank/points, claims airdropped points, redeems point rewards.
7. Buys Store Points or Bundles on `/bundles-points` to accelerate the above.
8. Disconnecting wallet (or switching accounts in the wallet extension) clears the session and locks gated pages again.

Admin

1. Navigates directly to `/admin` (no login screen exists today — see issues).
2. Creates a trait: fills form, sets USD price (auto-converted to ETH via live rate), supply, rarity, optional payout splits; toggles active to publish it to the storefront.
3. Uses bulk variant upload to add alternate skins for a themed drop, matched by filename to trait names.
4. Adjusts fees/store settings/appearance as needed; reviews stats and transactions for drop performance.
5. Manages the points economy: creates point packs/bundles, airdrops bonus points, curates bounty reward traits.

Bounty participant

1. Visits `/sandbox`, attempts to match the daily target look.
2. On match, gets a celebration + points; capped completions per day (`daily_bounty_completions`).
3. Spends accumulated points on `/bounties` reward traits.

Buyer/seller in Swap & Market

1. Seller lists a locker item (must be unequipped) on `/swap` (barter) or as a Market listing (fixed price).
 2. Buyer/trader browses listings, either buys outright (Market) or proposes a counter-offer of their own items (Swap).
 3. On acceptance, ownership of `locker_items` rows is reassigned atomically for all involved items; both parties get `transactions` log entries.
-

5. Database / Schema Breakdown

All tables live in one Postgres database (Drizzle ORM). `nftCollection` (`"wegens"` | `"wegenettes"`) is used throughout to keep the two collections' catalogs/settings separate within shared tables.

Table	Key Columns	Notes
<code>traits</code>	<code>id</code> (PK), <code>name</code> , <code>category</code> , <code>description</code> , <code>imageUrl</code> , <code>mediaType</code> , <code>priceUsd</code> , <code>priceEth</code> , <code>priceWei</code> , <code>totalSupply</code> , <code>remainingSupply</code> , <code>theme</code> , <code>dropName</code> , <code>isActive</code> , <code>rarity</code> , <code>nftCollection</code> , <code>payoutSplits</code> (jsonb array of <code>{walletAddress, percentage}</code>), <code>createdAt</code> , <code>updatedAt</code>	Core sellable item. <code>priceEth</code> / <code>priceWei</code> are always re-derived server-side from <code>priceUsd</code> at the live ETH rate.
<code>rarity_tiers</code>	<code>id</code> (PK), <code>name</code> , <code>nftCollection</code> , <code>rank</code> , <code>color</code>	Unique on (<code>name</code> , <code>nftCollection</code>); customizable rarity labels/colors per collection.
<code>trait_variants</code>	<code>id</code> (PK), <code>traitId</code> → <code>traits.id</code> (cascade), <code>name</code> , <code>imageUrl</code> , <code>mediaType</code> , <code>sortOrder</code> , <code>isEnabled</code>	Alternate skins for a trait, grouped by <code>name</code> into "packs".
<code>locker_items</code>	<code>id</code> (PK), <code>traitId</code> → <code>traits.id</code> , <code>walletAddress</code> , <code>quantity</code> , <code>equippedToTokenId</code> (nullable), <code>txHash</code> , <code>purchasedAt</code>	The user's inventory; <code>equippedToTokenId</code> links to <code>wegen_nfts.tokenId</code> when worn.
<code>wegen_nfts</code>	<code>tokenId</code> (PK), <code>walletAddress</code> , <code>name</code> , <code>imageUrl</code> , <code>metadataTxHash</code> , <code>metadataUpdatedAt</code> , <code>variantPack</code> , <code>createdAt</code>	Backend record of NFT ownership — NOT read from chain (see issues).
<code>store_settings</code>	<code>id</code> (PK), <code>fee %s</code> + <code>wallets</code> , <code>layerOrder</code> (json string), <code>storeName</code> / <code>Tagline</code> / <code>Open</code> , <code>announcementBanner</code> , <code>maxTraitsPerOrder</code> , <code>contractAddress</code> , <code>collectionWallet</code> , <code>networkName</code> , <code>social links</code> , <code>maintenanceMode</code> + <code>whitelist</code> (json string), <code>ineligibleNfts</code> (json string), <code>onChainUpdateFeeEth/Wallet</code> ,	One row per <code>nftCollection</code> ; created lazily on first access.

Table	Key Columns	Notes
	updateAuthorityKeyCiphertext, dailyGameEnabled/Overrides, celebration/logo/background/banner URLs, themeColors, nftCollection, updatedAt	
transactions	id (PK), type (enum buy/sell/trade), traitId → traits.id, traitName/ Category/ImageUrl (denormalized), walletAddress, ethAmount, ethPriceAtPurchase, txHash, tokenId, nftCollection, createdAt	Append-only activity log used by admin stats and recent activity.
swap_listings	id (PK), posterWallet, lookingFor, status (open/accepted/cancelled), acceptedByWallet, nftCollection, createdAt, updatedAt	
swap_listing_items	id (PK), listingId → swap_listings.id (cascade), lockerItemId → locker_items.id, traitId, traitName/ Category/ImageUrl	The items offered in a swap listing.
market_listings	id (PK), sellerWallet, lockerItemId → locker_items.id, traitId, traitName/ Category/ImageUrl, priceEth, status (active/sold/cancelled), buyerWallet, nftCollection, createdAt, updatedAt	
legends	id (PK), name, nftCollection, tokenId, imageUrl, mediaType, description, isActive, sortOrder, createdAt, updatedAt	1-of-1 special items.
legend_variants	id (PK), legendId → legends.id (cascade), name, imageUrl, mediaType, isEnabled, sortOrder	Alternate skins for a legend.
wallet_points	walletAddress (PK), totalPoints, updatedAt	Bounty/quest points balance.
point_transactions	id (PK), walletAddress, type (enum: purchase/confirm_traits/ sandbox_bounty/redeem/ admin_airdrop), points, description, claimedAt, createdAt	Audit log for wallet_points changes.
bounty_traits	id (PK), name, description, imageUrl, pointCost, totalSupply, remainingSupply, isActive,	Redeemable rewards; -1 supply = unlimited.

Table	Key Columns	Notes
	sourceTraitId (optional link to a real traits row), createdAt	
bounty_purchases	id (PK), walletAddress, bountyTraitId → bounty_traits.id, createdAt	Redemption history.
daily_bounty_completions	id (PK), walletAddress, completedDate, count	Rate-limits sandbox bounty completions per wallet per day.
point_packs	id (PK), name, description, imageUrl, usdValue, pointsGranted, isActive, createdAt, updatedAt	Store Points SKUs.
store_points	walletAddress (PK), totalPoints, updatedAt	Separate currency balance from wallet_points (bounty points vs. purchasable points).
store_point_purchases	id (PK), walletAddress, pointPackId → point_packs.id, pointsGranted, usdValue, ethAmount, ethPriceAtPurchase, txHash, createdAt	
trait_bundles	id (PK), name, description, imageUrl, priceUsd, priceEth, priceWei, totalSupply, remainingSupply, isActive, createdAt, updatedAt	Multi-trait packs sold as one SKU.
bundle_items	id (PK), bundleId → trait_bundles.id (cascade), traitId → traits.id, quantity	Contents of a bundle.
bundle_purchases	id (PK), walletAddress, bundleId → trait_bundles.id, ethAmount, ethPriceAtPurchase, txHash, purchasedAt	

Example records (illustrative):

```

traits: { id: 1, name: "Golden Crown", category: "Headgear", priceUsd: "25.00", priceEth: "0.1"
locker_items: { id: 10, traitId: 1, walletAddress: "0xabc...", quantity: 1, equippedToToken: "0x123..."
wegen_nfts: { tokenId: 4821, walletAddress: "0xabc...", name: "Wegen #4821", metadataTxHash: "0x456..."
swap_listings: { id: 3, posterWallet: "0xabc...", lookingFor: "Any rare Eyes trait", status: "open"

```

6. Backend / API Breakdown

Base path: `/api` (Express 5 app; session middleware, CORS, JSON body parsing, pino request logging).

Auth (`auth.ts`)

Method & Path	Input	Output	Called by
GET <code>/auth/nonce?address&chainId</code>	query	<code>{ nonce, message }</code>	WalletContext connect flow
POST <code>/auth/verify</code>	<code>{ address, message, signature }</code>	<code>{ success, walletAddress }</code>	WalletContext connect flow
GET <code>/auth/session</code>	—	<code>{ walletAddress }</code> or 401	WalletContext mount
POST <code>/auth/disconnect</code>	—	<code>{ success }</code>	WalletContext disconnect / account-changed

Traits & Store (`traits.ts`)

- GET `/traits/categories`, `/store/themes`, `/traits` (list w/ pagination + filters), `/traits/variant-collections`, `/traits/variants/by-collection`, `/traits/:traitId`, `/traits/:traitId/variants`, `/store/stats`, `/store/config` — all read-only, feed Store/Locker/Nfts/Sandbox.

Locker (`locker.ts`)

- GET `/locker/:walletAddress` — owned traits with trait detail joined.
- POST `/locker/:walletAddress/purchase` (auth: `requireWalletOwnership`) — buys a trait; body `{ traitId, txHash?, quantity? }`; converts USD→ETH server-side via `convertUsdToEth`; decrements supply; inserts locker item + transaction; awards points. Called by Store checkout.

NFTs (`nfts.ts`)

- GET `/nfts/:walletAddress` — NFTs + equipped traits per category.
- POST `/nfts/:tokenId/apply-trait` (auth) — body `{ lockerItemId, walletAddress }`; auto-unequips same-category conflicts.

- POST `/nfts/:tokenId/remove-trait` (auth) — body `{ category, walletAddress }`.
- POST `/nfts/:tokenId/confirm-traits` (auth) — body `{ walletAddress, variantPack? }`; generates a simulated tx hash, stamps `metadataTxHash` / `metadataUpdatedAt`, awards points. Called by Locker/Nfts SOC.

Admin (`admin.ts`) — no auth middleware anywhere in this file

- Traits: POST `/admin/traits`, PUT `/admin/traits/:traitId`, DELETE `/admin/traits/:traitId`.
- Fees: GET/PUT `/admin/fees`.
- Stats: GET `/admin/stats`, GET `/admin/transactions`.
- Layers: GET/PUT `/admin/layers`.
- Store settings: GET/PUT `/admin/store-settings` (+ appearance fields, maintenance, social links).
- Variant packs / bulk upload endpoints referenced from Admin.tsx (e.g. `/admin/variant-packs`, `/admin/traits/:traitId/variants`, `/admin/variants/:variantId`) — trait skin management.
- Legends admin CRUD + variants.
- NFTs admin list/delete.

Market (`market.ts`)

- GET `/market/listings?status&seller`, POST `/market/listings` (list an item), DELETE `/market/listings/:id` (cancel), POST `/market/listings/:id/buy` (executes sale, transfers `locker_items` ownership, logs two `transactions` rows).

Swap (`swap.ts`)

- GET `/swap/listings?wallet&status`, POST `/swap/listings` (create), DELETE `/swap/listings/:id` (cancel), POST `/swap/listings/:id/accept` (executes trade, reassigns items both directions, logs `trade` transactions).

Storage (`storage.ts`)

- POST `/storage/uploads/request-url` — returns a presigned upload URL + `objectPath` for direct client → object storage upload.
- GET `/storage/public-objects/*filePath`, GET `/storage/objects/*path` — serve stored assets.

Points (`points.ts`)

- GET `/point-packs` , GET `/wallet/:walletAddress/points` , POST `/point-packs/:packId/purchase` .
- Admin: GET/POST `/admin/point-packs` , PUT/DELETE `/admin/point-packs/:packId` .

Bundles (`bundles.ts`)

- GET `/bundles` , POST `/bundles/:bundleId/purchase` .
- Admin: GET/POST `/admin/bundles` , PUT/DELETE `/admin/bundles/:bundleId` .

Bounties (`bounties.ts`)

- GET `/bounties/leaderboard` , GET `/bounties/me` (session-based), POST `/bounties/sandbox-complete` , POST `/bounties/claim-points` , GET `/bounties/traits` , POST `/bounties/traits/:id/redeem` .
- Admin: POST `/admin/bounties/send-points` (airdrop), GET `/admin/bounties/traits` , POST `/admin/bounties/traits` , PATCH `/admin/bounties/traits/:id` , DELETE `/admin/bounties/traits/:id` , GET `/admin/bounties/leaderboard` , GET `/admin/bounties/point-log?page` .

Metadata (referenced in memory, not in task file list but part of the app)

- `/api/metadata/:collection/:tokenId[/image]` — serves ERC-721-style JSON metadata and a sharp-composited PNG image reflecting the NFT's currently equipped traits (used for on-chain metadata resolution / marketplaces).

Cross-cutting server logic

- `convertUsdToEth(usd)` / `getEthUsdRate()` (`lib/ethPriceService.ts`) — fetches a live ETH/USD rate from an external price source; throws `EthPriceUnavailableError` → routes return HTTP 503 rather than trusting a client-supplied ETH amount.
- `requireWalletOwnership()` / `requireAuth` middleware — ensures the `:walletAddress` route param or body field matches the session's authenticated wallet before allowing mutations on locker/NFTs/bounties/points/bundles.
- `awardPoints(wallet, amount, type, description)` (`bounties.ts`) — shared helper for granting `wallet_points` , called from `purchase/confirm-traits/`

sandbox flows; failures are swallowed (`.catch(() => {})`) so points issues never block the primary action.

7. Environment Variables & Outside Services

No secret values are included below — names only.

Variable	Purpose
<code>DATABASE_URL</code>	PostgreSQL connection string (Drizzle ORM pool)
<code>SESSION_SECRET</code>	Signs the Express session cookie and is used for the store-settings key-encryption helper
<code>PORT</code>	Port the API server binds to
<code>NODE_ENV</code>	Toggles secure cookies + logger verbosity
<code>LOG_LEVEL</code>	Pino logger verbosity
<code>PUBLIC_OBJECT_SEARCH_PATHS</code>	Object storage: search paths for public asset serving
<code>PRIVATE_OBJECT_DIR</code>	Object storage: directory/bucket prefix for uploaded assets

External services relied on: - **PostgreSQL** — primary datastore. - **Object storage (S3/R2-compatible)** — trait images, uploaded media, appearance assets, served via presigned URLs. - **ETH/USD price feed** — external HTTP price source used server-side to convert USD-denominated prices to ETH at purchase/creation time (source is abstracted behind `ethPriceService`; a real deployment should confirm which price API is used — do not trust the frontend for this rate). - **EVM wallet providers** — MetaMask, Phantom, Backpack, Coinbase Wallet, OKX, Trust Wallet, Rabby, Rainbow, Brave, or any `window.ethereum`-injecting wallet, accessed client-side via `ethers.js` — no server-side wallet SDK. - **Image compositing (server-side)** — `sharp` (or equivalent) composites layered trait images into a flattened NFT preview PNG for the metadata image endpoint.

No third-party auth provider, payment processor, or email service is used — purchases are simulated (there is no real on-chain settlement; a "tx hash" is either a real wallet-returned hash passed through, or a randomly generated 64-hex-character placeholder for internal actions like SOC).

8. Design & UI Notes

- **Stack:** React + Vite + TailwindCSS (v4, HSL CSS-variable tokens) + shadcn/ui components (Dialog, Sheet/cart drawer, Tabs, Select, Table, Card, Badge, Switch, Toast).
 - **Visual identity:** dark, "cyberpunk arcade" aesthetic — near-black backgrounds with a strong purple (`hsl(272 100% 62%)`) primary and gold accent (`hsl(43 100% 52%)`) filigree borders on headers/panels. Ambient ammo: drifting radial-gradient "orbs" and rising "particle" sparks layered behind content for atmosphere.
 - **Dual-collection theming:** switching the active collection (Wegens ↔ Wegenettes) live-updates CSS variables — Wegenettes uses hot pink (`hsl(320 100% 55%)`) as primary with purple as a secondary accent, plus its own background gradient and its own trait/NFT data set.
 - **Typography:** Rajdhani for body/UI text, Bebas Neue for headings, Bungee for the main logo/title, Rubik Spray Paint / Permanent Marker for graffiti-style flourishes (tags, wallet address chip).
 - **Rarity badges:** color-coded chips (grey=common, green=uncommon, blue=rare, gold/legendary=orange-gold) shown on every trait card.
 - **Cards:** consistent dark translucent panel with a subtle border and hover glow, used for traits, locker items, NFTs, bounty rewards, and admin list rows.
 - **Modals:** Dialog for equip/SOC confirmation and admin create/edit forms; a Sheet slide-over for the shopping cart.
 - **Admin dashboard layout:** tabbed single-page layout (Tabs / TabsList) — Traits / Legends / NFTs / Stats / Fees / Settings / Appearance / Points / Bounties / Bundles — each tab renders its own table + create/edit dialog pattern, consistent with the storefront's dark theme (not a separate light "admin" skin).
 - **Mobile behavior:** primary nav becomes a horizontally scrollable icon+label belt (`overflow-x-auto` , hidden scrollbar) rather than a hamburger menu; cards reflow to a single/double column grid.
 - **Iframe/preview warning banner:** the app detects when it's being viewed inside an iframe (e.g. a dev preview) and shows a banner, since wallet extensions refuse to inject into iframes — this is a real, load-bearing UX affordance, not decorative.
-

9. Current Issues / Broken or Incomplete Parts

Verified by reading the code (not guessed):

- Admin panel has no authentication.** `artifacts/api-server/src/routes/admin.ts` defines every `/admin/*` route with no auth middleware, session check, or role gate — anyone who knows/guesses the `/admin` URL can create/edit/delete traits, change fees, airdrop points, etc. **Correct behavior:** gate the entire `/admin` route (frontend) and every `/admin/*` API route (backend) behind an authenticated admin session (e.g. an allowlisted wallet address, or a separate admin login), matching the pattern already used for wallet-owned user routes via `requireWalletOwnership`.
- NFT ownership is not verified on-chain.** `wegen_nfts` is a backend-only table (no reads from the actual NFT contract/RPC node). Anything in this table is trusted as-is — there is no proof the `walletAddress` actually holds that `tokenId` on-chain. **Correct behavior:** verify ownership via an on-chain read (e.g. `ownerOf(tokenId)` against the real collection contract) before allowing trait purchases/equips tied to a token, or at minimum before trusting `wegen_nfts` rows created by anything other than a verified on-chain indexer.
- Purchases and on-chain updates are simulated, not real blockchain transactions.** `confirm-traits` and several purchase flows generate a random 64-hex-character string as a stand-in `txHash` rather than broadcasting a real transaction and waiting for a receipt. This is acceptable for a prototype/demo but is not production-ready for a real paid marketplace. **Correct behavior:** integrate real contract calls (mint/transfer/metadata update) and verify transaction receipts server-side before crediting purchases.
- In-memory SIWE nonce store does not survive server restarts or multi-instance deployments.** `auth.ts` stores nonces in a plain `Map` in process memory. On restart, all in-flight sign-in attempts fail; in a horizontally-scaled deployment, a request routed to a different instance than the one that issued the nonce will always fail verification. **Correct behavior:** back the nonce store with Redis/Postgres (with the existing 5-minute TTL), or a stateless verifiable nonce scheme.
- NFT demo wallet is hardcoded.** Per project notes, the seeded NFT wallet address `0xDEMOwallet` is a placeholder for demo purposes and is not a real address — the app has never been tested against a live NFT contract/wallet pairing at scale.

No other functional defects were identified from the code reviewed (route handlers consistently validate input with Zod, use transactions/ownership checks appropriately, and return meaningful error codes such as 400/403/404/503).

10. Fresh Build — Phased Plan

Phase 1 — Foundation - Set up the chosen stack's project skeleton (frontend framework, backend framework, routing, build tooling). - Establish the design system: dark theme tokens, typography, rarity badge components, card components. - Set up environment variable / secrets management per Section 7.

Phase 2 — Database - Create every table in Section 5 with the given columns, types, defaults, and foreign keys (respect cascade deletes where noted). - Seed at least one `store_settings` row per collection and a handful of demo traits/NFTs for local development.

Phase 3 — Authentication - Implement SIWE-style wallet auth: nonce issuance, message signing, signature verification, session issuance (use a durable session/nonce store, not in-memory, learning from Issue #4). - Implement multi-wallet detection (EVM only — do not add a parallel Solana/other-chain connect flow) and the account/chain-change listeners. - Add wallet-ownership middleware for any route acting on a specific `walletAddress`. - Add real admin authentication (fixing Issue #1) before any admin route is reachable.

Phase 4 — Core pages - Build Store (browse/filter/cart/checkout), Locker (equip/unequip/SOC), My NFTs (per-NFT equip management), matching the page-by-page spec in Section 3.

Phase 5 — Core features - Trait Swap & Market (listings, accept/buy/cancel flows with atomic ownership transfer). - Sandbox (try-on, randomize, save PNG, daily bounty). - My Legends (variant toggle, OpenSea link). - Bounties/points economy (leaderboard, claim, redeem) and Bundles/Points purchase page. - Wire USD→ETH pricing through a single server-side price service (never trust client-submitted ETH amounts, per the existing pattern).

Phase 6 — Admin tools - Build the tabbed admin dashboard: Traits, Legends, NFTs, Stats, Fees, Store Settings, Appearance, Layers, Points, Bounties, Bundles, Transactions log — behind the admin auth from Phase 3. - Build bulk variant upload (filename-matching) and variant pack enable/disable tooling.

Phase 7 — Testing - Unit/integration tests for purchase, equip/unequip, swap/market transfer atomicity, and points awarding. - End-to-end tests for the full connect → buy → equip → SOC flow and the admin CRUD flows. - Manual verification of mobile nav behavior and the iframe-preview warning.

Phase 8 — Launch polish - Real on-chain integration (replace simulated tx hashes) if going to production with real money, per Issue #3. - On-chain NFT ownership verification, per Issue #2. - Rate limiting, error monitoring, and a durable nonce/session store, per Issue #4. - Final accessibility, mobile, and performance pass.

11. Copy-Ready Prompt for Another AI Builder Tool

Build a full-stack Web3 NFT trait marketplace called "Wegen NFT Trait Store" (a game-cosmetic

CONCEPT

NFT holders connect an Ethereum wallet, browse purchasable cosmetic "traits" (Background, B

TECH EXPECTATIONS

- Full-stack TypeScript. React frontend + Node/Express (or equivalent) backend + PostgreSQL
- Wallet auth via Sign-In-With-Ethereum (SIWE): issue a nonce, have the wallet sign a messa
- All trait prices are entered as USD by the admin and converted to ETH server-side at purch
- Every write endpoint must validate input against a schema (e.g. Zod) and return proper 400
- Protect every endpoint that acts on a specific wallet's data with ownership middleware th
- **The admin panel and every admin API route MUST require real authentication (admin login
- Use a durable (not in-memory) store for SIWE nonces and sessions so auth survives restart

DATA MODEL

Implement these tables (Postgres): traits, rarity_tiers, trait_variants, locker_items, wege

PAGES & FEATURES

1. Store (`/`) – browse/filter traits by category+theme, rarity badges, cart, checkout with
2. Trait Locker (`/locker`) – view owned traits, filter/sort, equip to a chosen owned NFT (
3. My NFTs (`/nfts`) – per-NFT gallery showing equipped traits by category, equip-picker di
4. Trait Swap & Market (`/swap`) – post/cancel barter listings with offered items + "lookin
5. Sandbox (`/sandbox`) – try on any trait without owning it, randomize, clear, download a
6. My Legends (`/my-legends`) – view owned 1-of-1 legend NFTs, toggle between enabled cosme
7. Bounties (`/bounties`) – leaderboard, personal rank/points/history, claim pending points
8. Packs & Points (`/bundles-points`) – buy Store Points currency in packs, buy multi-trait
9. Admin (`/admin`, auth-gated) – full CRUD for traits (incl. bulk variant upload matched b

DESIGN

Dark "cyberpunk arcade" theme: near-black background, purple primary (~#8833ff / hsl(272,100

BUILD ORDER

1. Foundation (project scaffold, design tokens, env/secrets).
2. Database (all tables above).
3. Authentication (SIWE wallet auth + durable session/nonce store + admin auth).
4. Core pages (Store, Locker, My NFTs).
5. Core features (Swap/Market, Sandbox, My Legends, Bounties, Packs & Points) with a single
6. Admin tools (full dashboard, gated behind real admin auth from step 3).
7. Testing (purchase/equip/trade atomicity, e2e connect-to-equip flow, admin CRUD).
8. Launch polish (real on-chain settlement if going to production with real funds, on-chain

Build this iteratively, starting with the data model and auth, then the Store/Locker/NFTs l